Problem Situation - Evidence 2

Aldo Karim Garcia Zapata
Departamento de Computación, Tecnológico de Monterrey
Monterrey, México
A00843726@tec.mx

***Abstract*—Botnet attacks have multiplied to the point of causing major disruptions across all sectors [1]. A typical trace of these attacks is a server log with thousands of failed access attempts coming from many IP addresses. This report describes a C++ solution that reads a log of 16,807 records, stores them in a doubly linked list, sorts them by IP address and date/time with an iterative Quicksort that works directly on the list with its own stack, and extracts the records of an IP range given by the user, which are printed and saved to a file. The chosen data structures and algorithms are justified, their best- and worst-case complexity is analyzed and tested, the suitability of the linked list is discussed, and improvements are proposed.**

***Index Terms*—botnet, log file, doubly linked list, stack, Quicksort, range search, computational complexity.**

I. WHAT IS THE PROBLEM, AND WHY IS IT RELEVANT?

A. The Problem

According to the FBI [1], cyberattacks carried out with botnets have multiplied to the point of causing major disruptions across all sectors. A botnet is a network of infected devices that an attacker controls remotely to launch coordinated actions, such as brute-force attacks against remote access services (SSH).

Each attempt leaves a line in the server log with this format:

Oct 9 10:32:24 423.2.230.77:6166 Failed password for illegal user guest

The fields are: month, day, time, IP:port, and message. The file contains 16,807 records in random order, so it is impossible to tell at a glance:

- Which IP tried to access the most times (possible bots).
- The chronological sequence of attempts from a single IP.
- Efficient lookup of the records of one IP or a range of IPs.

B. Relevance

Detection: an IP with hundreds of consecutive failures within seconds is a classic indicator of a bot. Grouping by IP and sorting by time makes it evident.

Incident response: forensic analysis needs an ordered timeline to know when the attack began and which accounts were targeted (root, admin, guest).

Scale: real logs grow to millions of lines, so the choice of data structures and algorithms directly affects analysis time.

Current context: law enforcement operations against botnets show that the problem is persistent and that many always remain active [1].

II. HOW WAS THE PROBLEM SOLVED?

The solution is divided into five modules plus main.cpp, which calls them in order (Fig. 1):

Module	File	Responsibility
Record	Entry.h/.cpp	Models a log line and computes its numeric timestamp.
Keeper	Keeper.h/.cpp	Doubly linked list that reads and stores the records.
Sorter	Sorter.h/.cpp	Sorts the list in place by IP and then by date/time (iterative Quicksort).
Stack	Stack.h/.cpp	Singly linked list stack that replaces recursion in the Quicksort.
Searcher	Searcher.h/.cpp	Asks for an IP range, finds its records, prints them and saves them to output.txt.

main.cpp reads bitacora.txt into the list, prints how many records were read, sorts the list by IP and date, asks the user for an IP range, and finally prints the records of that range and saves them to output.txt.

III. DATA STRUCTURES AND ALGORITHMS

A. Doubly Linked List (Keeper)

Implementation: each Node stores an Entry data and next and prev pointers. The Keeper class maintains head, tail, and count (Fig. 2). Keeper::read takes the month, day, time and IP of each line with >> and the rest with getline as the message, and inserts the record at the end with pushBack.

Why it was chosen:

- Records arrive as a stream and their total number is

unknown at the start; the list grows without reserving or copying memory.

- By storing tail, inserting at the end is truly O(1) (no list traversal).
- The prev pointer gives the end of the left partition in the Quicksort (pivot->prev) in O(1), allows traversal in both directions, and allows deleting a node in O(1) given its pointer.
- It is the main change from Evidence 1, where two vectors were used; the struct keeps each record's data together.

B. Entry (record struct)

Stores the original fields (to reprint them exactly with toString) and a numeric timestamp = month·10^8 + day·10^6 + hour·10^4 + minute·10^2 + second, so two dates are compared with a single integer comparison. It also defines operator< by date, but the Sorter uses its own comparison (lessThan) because the first key is the IP.

C. Iterative Quicksort on the List (Sorter)

Implementation (Fig. 3):

- It works directly on the doubly linked list: each pending partition is delimited by two node pointers (low, high) instead of two indices, so no copy of the records is made.
- Lomuto partition [2]: the last node (high) is the pivot. j walks from low to the node before high and i marks the last node smaller than the pivot; when j is smaller, i advances and the data of i and j is swapped. At the end the pivot is swapped into the node after i, its final position.
- Only the data of the nodes is swapped: the next and prev pointers never change, so the list stays connected.
- Comparison (lessThan): first by IP and, if the IPs are equal, by timestamp. ipToNumber turns the IP into a base-1000 number, because comparing “10.x” with “9.x” as text gives a wrong order and the log contains octets greater than 255 (e.g., 423.2.230.77), which would overlap in base 256. The port is ignored.
- Instead of recursion, the pending partitions are stored in our own Stack (Section III-D). After each partition, the left side (low to pivot->prev) and the right side (pivot->next to high) are pushed if they are not empty.

```
Node *partition(Node *low, Node *high) {
   Node *i = low->prev;
   for (Node *j = low; j != high; j = j->next)
      if (lessThan(j->data, high->data)) {
         i = (i == nullptr) ? low : i->next;
         swap(i->data, j->data);
      }
   i = (i == nullptr) ? low : i->next;
   swap(i->data, high->data);
   return i;
}
```

Why it was chosen:

- Quicksort: very fast in practice, sorts in place and is the algorithm covered in class.
- Iterative: It is a requirement and avoids overflowing the call stack with deep recursion.
- Last node as pivot (Lomuto): the partition only moves forward through the list (j = j->next), which a linked list does well. Choosing the middle node would need an extra traversal.
- In place on the list: the 16,807 records are not copied to another structure; the only extra memory is the stack.

D. Stack (Stack.h/.cpp)

Implementation: a singly linked list where the top of the stack is the head. Each StackNode stores two pointers to list nodes (low and high), which mark a partition still to be sorted, and a next pointer. It replaces the recursion of the Quicksort: each element is a pending recursive call.

- push(low, high) inserts a node at the head: top = new StackNode(low, high, top).
- pop(low, high) returns the range of the top through references, removes and frees that node, and returns false when the stack is empty; that false ends the sorting loop.
- isEmpty() checks top == nullptr, and the destructor frees any node left.

E. Range Search (Searcher.h/.cpp)

Implementation (Fig. 4). Because the list is sorted by IP and then by date, the records of the range are together and already in chronological order, so the result is a segment of the list (first to last):

- askRange asks for the start and end IP. ipToKey validates each one (four octets of up to three digits, with an optional port) and converts it to the same base-1000 number used by the Sorter; an invalid IP is asked again. If the start is greater than the end, both are swapped and the user is notified.
- search skips the nodes whose IP is smaller than the start, then advances while the IP is not greater than the end, saving the first and last nodes of the range and the count. It stops as soon as it passes the end IP.
- display prints the count records starting at first with toString (Month Day Hour:Minute:Second IP:port message), and store writes the same records to output.txt.

IV. IS A LINKED LIST APPROPRIATE FOR THIS PROBLEM?

Yes for reading and sorting the log, but it limits the search and the choice of pivot.

In favor: the number of records is unknown, and pushBack adds each one in O(1) without reallocating or copying. It also fits the algorithms used: Lomuto partition only needs sequential access, pivot->prev gives the end of the left partition in O(1), and the search result is a segment of the list that is printed and saved without copying records.

Against: there is no random access. The middle node cannot be chosen as pivot without an extra traversal, so the last node is used and a sorted or reversed input gives the $O(n^2)$ worst case (Section VI); and binary search is impossible, so the range search is O(n) in the worst case instead of O(log n). For intensive searching a dynamic array would be faster, and a merge sort on the list would avoid that worst case. With 16,807 records the difference is small; with millions it would be noticeable.

V. ADVANTAGES AND DISADVANTAGES

Structure / algorithm	**Advantages**	**Disadvantages**
Doubly linked list	O(1) insertion at the end; grows without copying; allows sorting in place and returning the search result without copies.	O(n) access by position, so no binary search; two extra pointers per node; worse cache locality than an array.
Iterative Quicksort	Average O(n log n); sorts in place on the list with only sequential access; no recursion, so no call stack overflow.	$O(n^2)$ when the input is already sorted or reversed; not stable (not a problem: the timestamp is the second key); the stack can grow to O(n).
Stack	push, pop and isEmpty in O(1); grows as needed, with no fixed capacity to overflow.	An extra next pointer and one dynamic allocation per element.
Range search	Simple; stops as soon as it passes the end IP; returns first and last without copying records; validates the IPs and fixes a reversed range.	O(n) in the worst case, since binary search is not possible on a list; only correct if the list was sorted first.

VI. COMPUTATIONAL COMPLEXITY

Operation	**Best case**	**Average case**	**Worst case**	**Extra space**
pushBack	O(1)	O(1)	O(1)	O(1)
read (full read)	O(n)	O(n)	O(n)	O(1) in addition to the list
monthToNumber / Entry() / toString	O(1)	O(1)	O(1)	O(1)
ipToNumber / ipToKey	O(1)	O(1)	O(1) (bounded IP length)	O(1)
Destructor (Keeper)	O(n)	O(n)	O(n)	O(1)
Stack push / pop / isEmpty	O(1)	O(1)	O(1)	O(1)
partition (k nodes)	O(k)	O(k)	O(k)	O(1)
Sorter::sort (iterative Quicksort)	O(n log n)	O(n log n)	$O(n^2)$	O(log n) to O(n) (stack)
Searcher::search	O(1)	O(p + m)	O(n)	O(1)
display / store	O(m)	O(m)	O(m)	O(1)

n: number of records; k: nodes in a partition; p: nodes before the start IP; m: records found.

A. What Determines Quicksort's Best and Worst Case?

The complexity depends on how balanced the partition is around the pivot:

- Best case O(n log n): the pivot splits the range into two nearly equal halves.
- Worst case $O(n^2)$: the pivot always lands at an extreme (the smallest or largest), producing partitions of size n-1 and 0. Since the pivot is the last node, this happens when the list is already sorted or in reverse order.
- Average case O(n log n): with data in arbitrary order, like bitacora.txt, the partitions are reasonably balanced.

Experiment: with the original bitacora.txt (random order) the whole program runs in about 0.02 s. With a copy of the log that was already sorted it takes about 4.6 s on the same computer, more than 200 times slower, which confirms the $O(n^2)$ worst case of the last-node pivot.

B. Why the Stack Is O(log n) to O(n)

Both subranges are pushed without first choosing the smaller one. If the partitions are balanced, the stack holds O(log n) ranges; if they are highly unbalanced (worst case), it can reach O(n).

C. What Determines the Range Search's Best and Worst Case?

- Best case O(1): the end IP is smaller than the first IP of the list, so both loops stop at the first node.
- Worst case O(n): the range reaches the last IPs of the list (or covers all of it), so every node is visited.
- In general it is O(p + m): the p nodes before the start IP are skipped and the m records of the range are collected; display and store are O(m).

VII. POSSIBLE IMPROVEMENTS

- Bot detection: once sorted, a single O(n) pass counts attempts per IP and flags those exceeding a threshold within a time window.
- Top offenders ranking: list the N IPs with the most failed attempts, with their first and last attempt times.
- Graphical visualization: a chart (bar or heat map) of attempts per IP or per hour, exported from the program.
- Store the numeric IP in Entry when the file is read, so Sorter and Searcher do not convert the IP text on every comparison, and use a single conversion function instead of ipToNumber and ipToKey.

VIII. OPERATION FLOW DIAGRAMS

Figs. 1–4 show the flow of the whole program and of each main operation.

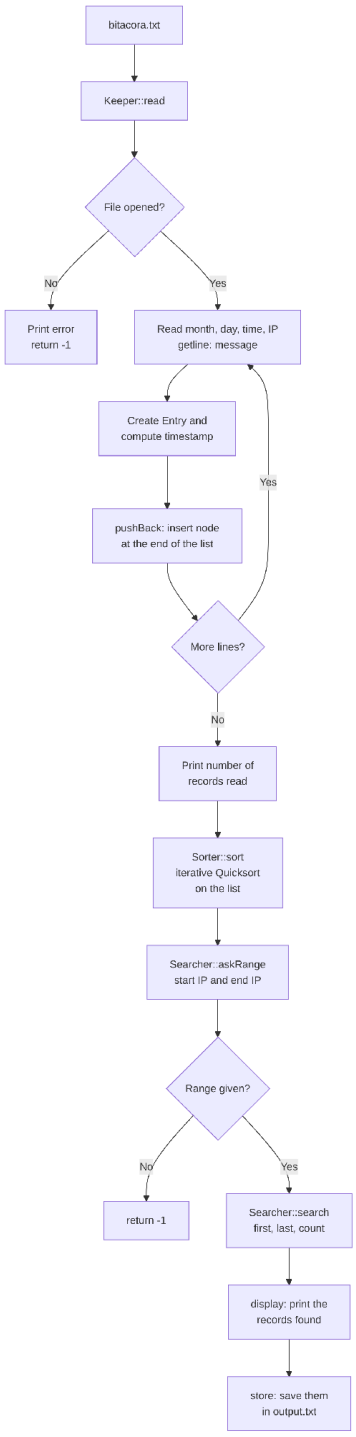


Fig. 1. General program flow.

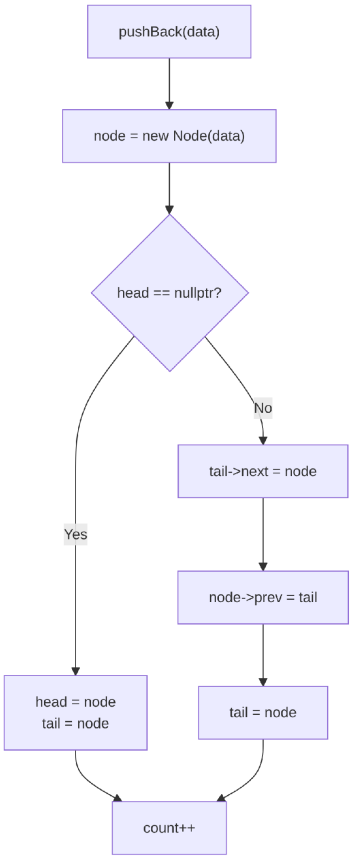


Fig. 2. pushBack operation on the doubly linked list.

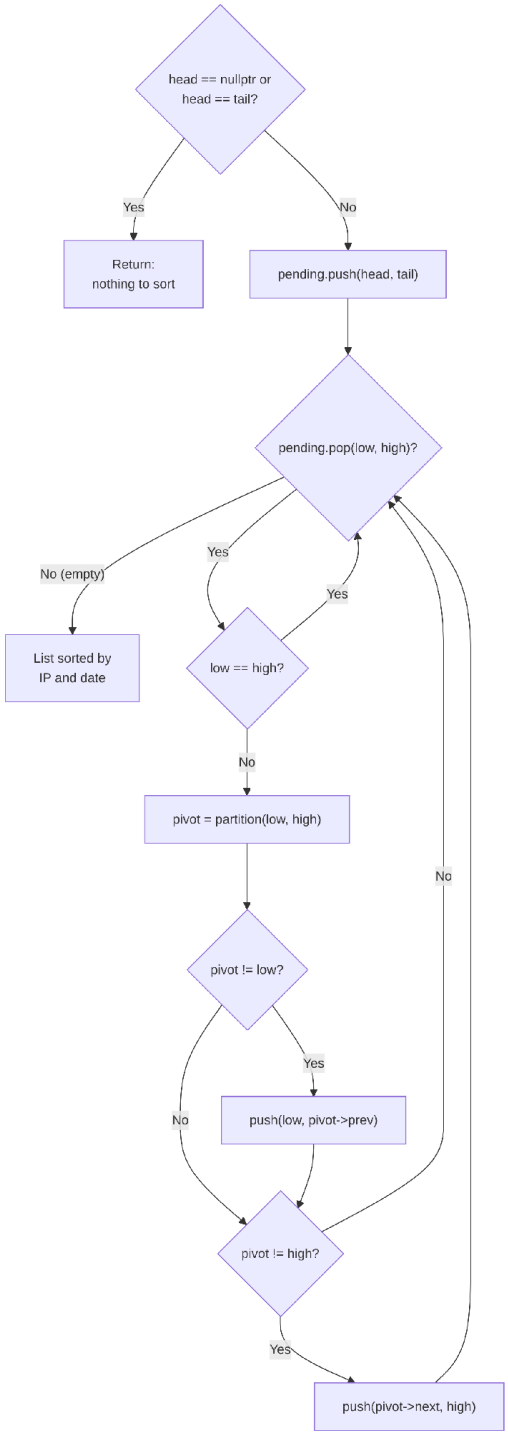


Fig. 3. Iterative Quicksort with the Stack of pending partitions.

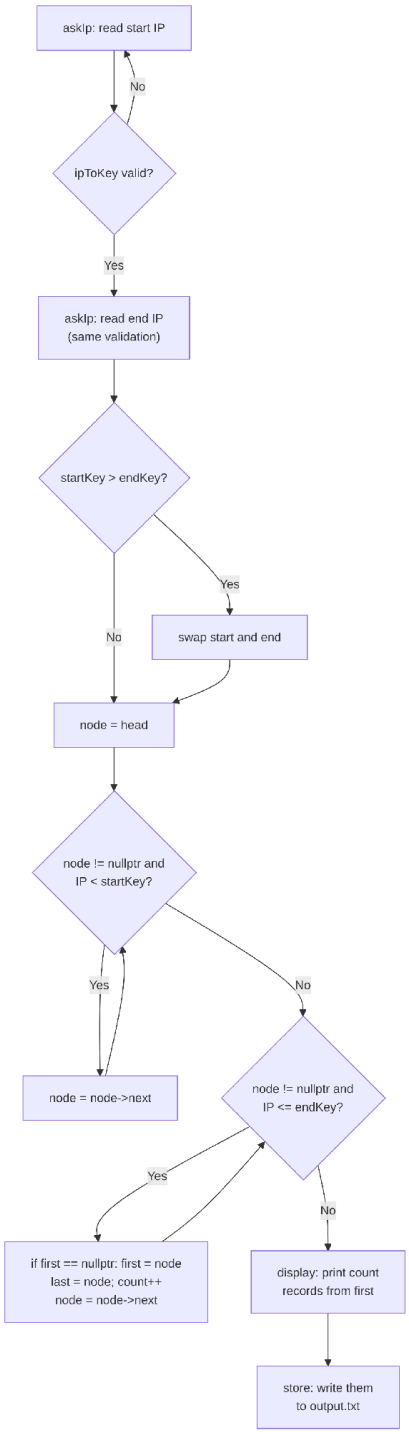


Fig. 4. Range search, display and store.

IX. CONCLUSIONS

The combination of a doubly linked list to read the log, an iterative Quicksort that sorts the list in place with the help of our own stack, and a linear range search makes it possible to sort the 16,807 records by IP and date and extract the attempts of any IP range, which helps identify botnet attack patterns. The list offers O(1) insertion and dynamic growth, and lets the search return a segment without copying records, but its lack of random access rules out binary search and forces an extreme pivot. Sorting is O(n log n) on average and $O(n^2)$ in the worst case, which appears when the input is already sorted, as the experiment showed; the search is O(n) in the worst case. The most valuable next steps are a merge sort or a random pivot to remove that worst case, and storing the numeric IP to speed up the comparisons.

REFERENCES

[1] P. H. O'Neill, "The FBI shut down a huge botnet, but there are plenty more left," MIT Technology Review. [Online]. Available: https://www.technologyreview.com/s/604138/the-fbi-shut-down-a-huge-botnet-but-there-are-plenty-more-left/. [Accessed: Oct. 15, 2026].

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

AI usage: I used AI for better translating from Spanish to English and helping me to do the flowchart code.